Quantum Traffic Gridlock — Technical Report

Team: Prakhar Keshniya, Harsh Sahu, Pranav Keshniya · Code, outputs and run instructions: https://github.com/Harshu3605/quantum-traffic-gridlock-qaoa

1. The problem in one paragraph

Two linked intersections, **A** and **B**, each choose one green phase per 30-second slot for 4 slots: **1 = North–South (NS)** or **0 = East–West (EW)**. That gives 8 decision bits, ordered A1–A4 then B1–B4, and 2^8 = 256 possible plans. Vehicles that A serves on the linked approach arrive at B in the next slot, so the two decisions interact. The official cost is **delay + w_o·overflow + w_s·switches**: queued vehicles after each slot, plus a penalty for queues above storage and for each phase change. All costs in this report come from the organisers' unmodified `official/scorer.py`. It reproduces the reference values 116, 146, 237 and 92.5.

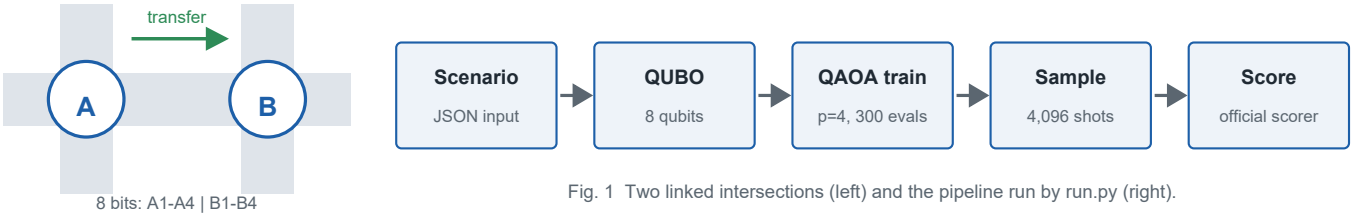


Fig. 1 Two linked intersections (left) and the pipeline run by run.py (right).

2. From traffic rules to a QUBO

QAOA needs a **QUBO**: a cost that contains only single bits and pairs of bits. We build it in three steps, using the same rule for every scenario.

Step 1: split the cost exactly. The total cost is the sum of four lane costs (A-NS, A-EW, B-NS, B-EW) plus the switch cost. A lane's queue depends only on the green bits that control it, which is 4 bits. The exception is B's linked lane, which also depends on A's 4 bits, because A feeds it. We checked that this split is exact for all 256 plans.

Step 2: switch cost, exactly quadratic. A change between slots is $x_t \oplus x_{t-1} = x_t + x_{t-1} - 2 \cdot x_t \cdot x_{t-1}$, which is already a QUBO term.

Step 3: lane costs, best quadratic fit. Queue cost is not quadratic. "At most 5 cars served" and "a queue never goes below 0" create higher-order effects. For each lane we keep the **best possible approximation that uses only single bits and pairs**: the order-2 Walsh–Fourier expansion of that lane's cost. Under equal weighting of all bit patterns, this is the least-squares best quadratic, so the fit does not favour any particular plan. Adding the four fitted lanes and the exact switch term gives the final QUBO.

Why this is allowed. The rule uses only traffic quantities: arrivals, capacity, storage, transfer and weights. It never searches for, ranks or encodes the best plan. Full enumeration appears only in the benchmarks. To be transparent, there is one caveat. Because B's linked lane depends on all 8 bits, fitting that one lane evaluates its queue for all 256 patterns. The result is still a fixed approximation rule that does not know the answer.

Size. 8 variables become 8 qubits, with no auxiliary qubits. The QUBO is converted to Ising form with $x = (1 - z)/2$, giving 8 Z terms and 28 ZZ terms.

3. QAOA design

- **Circuit.** Start with all plans equally likely ($|+\rangle^8$). Then repeat **p = 4** times (the maximum allowed): a cost layer (RZ and RZZ gates) that favours cheap plans, followed by a mixing layer (RX gates).
- **Training.** A classical optimiser (COBYLA) tunes the 8 angles to minimise the **average QUBO cost**, under a hard limit of **300 evaluations**. Lift is an average-cost metric, so this objective matches the judging metric. The cost is rescaled to unit spread, so the same angle range works for every scenario.
- **Warm start (INTERP).** Solve depth 1 first. Then stretch those angles to depth 2, re-optimise, and repeat up to depth 4. This turned out to be the key design choice (Fig. 3).
- **Sampling.** The trained circuit is run for exactly **4,096 shots** with a fixed seed. It is exported as `circuit.qasm`. We checked that re-loading the QASM reproduces the same distribution.
- **Cost.** About 2 seconds per run on a CPU, far below the 10-minute limit.

4. Results (official costs, 5 seeds each)

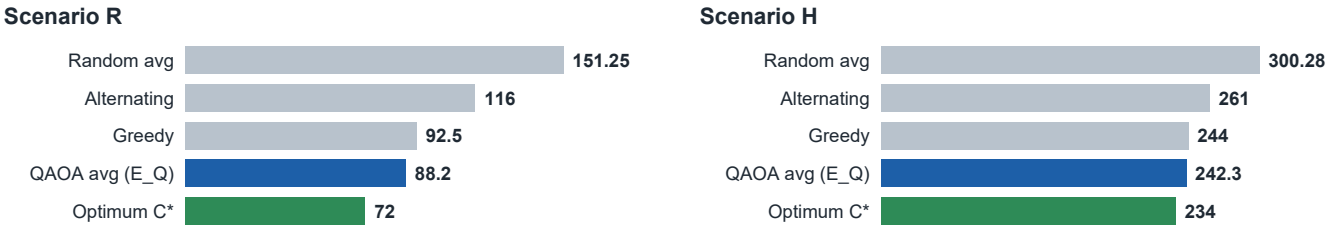


Fig. 2 Official cost of each method (lower is better). Blue = mean of 4,096 raw QAOA samples, averaged over 5 seeds.

	Scenario R	Scenario H
Random plan, average (C_rand)	151.25	300.28
Best alternating plan	116	261
Greedy (serve the longer queue)	92.5	244
Best plan of our QUBO (regret vs. optimum)	72 (0 %)	236 (0.9 %)
Raw QAOA, average over 4,096 shots (E_Q)	**88.2**	**242.3**
True optimum, from enumeration (C*)	72	234
Lift (mean / worst seed)	0.796 / 0.792	0.875 / 0.873
Elite enrichment (top-13 plans vs. random)	12.6×	16.9×
Shots that hit the optimum	8.3 %	11.5 %
Hybrid: QAOA + local search (labelled, not raw)	72	234

What this means. On both scenarios, an average QAOA sample is better than every simple rule, greedy included. QAOA closes about 80–88 % of the gap between random and optimal (the lift), and samples the 13 best plans 12–17 times more often than random. The most frequent H plan, 1001|0111, costs 235, only 1 above the optimum.

Generalisation. We generated 30 random scenarios from the published hidden ranges.

- QUBO regret averaged 0.9 %. It was at most 5 % on 28 of the 30 scenarios, and the worst case was 10.6 %.
- Mean lift was 0.84. Mean enrichment was 12.3×.
- Greedy's mean regret was 5.4 %.

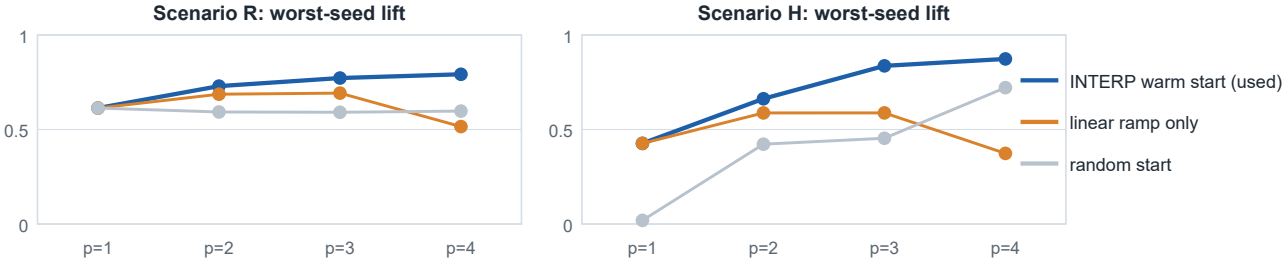


Fig. 3 Ablation over depth and initialisation (5 seeds, 300-evaluation budget). Only INTERP keeps improving as p grows.

Ablations. Only the INTERP warm start improves steadily with depth (Fig. 3). With plain or random starts, 300 evaluations are not enough at p = 3–4, and results become unstable. Changing the overflow penalty multiplier between 0.5 and 2 moved lift by less than 0.02. We kept the official weights and did not tune them on R or H.

5. Limitations

- The QUBO is an approximation. On heavily saturated traffic its best plan can be up to about 10 % worse than optimal. Auxiliary qubits could reduce this, but we did not implement it.
- Training uses an exact, noiseless simulator. We did no noise or hardware study.
- With 256 plans, a classical computer finds the optimum instantly, so we claim **no quantum advantage**. The contribution is a correct, reproducible pipeline whose samples are strongly concentrated on good plans.